RAJALAKSHMI ENGINEERING COLLEGE

Vaibhav Kiran R 251801363-2025-AIDS V2

Started on	Tuesday, 25 August 2026, 12:36 PM
State	Finished
Completed on	Tuesday, 25 August 2026, 1:14 PM
Time taken	37 mins 57 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void function (int n)
{
    int i= 1;
    int s =1;
    while(s <= n)
    {
         i++;
         s += i;
     }
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

For example:

Input	Result
9	12

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

void function(int n)
{
    int count = 0;

    int i = 1;
    count++; // Execution of: i = 1

    int s = 1;
    count++; // Execution of: s = 1

    count++; // First evaluation of while condition (s <= n)
    while (s <= n)
    {
        i++;
        count++; // Execution of: i++

        s += i;
        count++; // Execution of: s += i

        count++; // Re-evaluation of while condition (s <= n)
    }

    printf("%d\n", count);
}

int main()
{
    int n;
    if (scanf("%d", &n) == 1) {
```

```
32          function(n);
33      }
34      return 0;
35  }
```

	Input	Expected	Got	
✔	9	12	12	✔
✔	4	9	9	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Vaibhav Kiran R 251801363-2025-AIDS

V2

Started on	Thursday, 27 August 2026, 11:15 AM
State	Finished
Completed on	Thursday, 27 August 2026, 11:15 AM
Time taken	45 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

Example 1:

```
Input: nums = [3,2,3]
Output: 3
```

Example 2:

```
Input: nums = [2,2,1,1,1,2,2]
Output: 2
```

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 10^4`
- `-2^31 <= nums[i] <= 2^31 - 1`

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int nums[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &nums[i]);
    }

    int candidate = nums[0];
    int count = 1;

    for (int i = 1; i < n; i++) {
        if (nums[i] == candidate) {
            count++;
        } else {
            count--;
        }

        if (count == 0) {
            candidate = nums[i];
            count = 1;
        }
```

```
27      }
28
29      printf("%d", candidate);
30
31      return 0;
32  }
```

	Input	Expected	Got	
✔	3 3 2 3	3	3	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Vaibhav Kiran R 251801363-2025-AIDS

V2

Started on	Thursday, 27 August 2026, 11:17 AM
State	Finished
Completed on	Thursday, 27 August 2026, 11:19 AM
Time taken	1 min 41 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int findFloor(int arr[], int low, int high, int x) {
    if (low > high)
        return -1;

    int mid = low + (high - low) / 2;

    if (arr[mid] == x)
        return arr[mid];

    if (arr[mid] > x)
        return findFloor(arr, low, mid - 1, x);

    int result = findFloor(arr, mid + 1, high, x);

    if (result == -1)
        return arr[mid];

    return result;
}

int main() {
    int n, x;

    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &x);

    int floor = findFloor(arr, 0, n - 1, x);

    printf("%d", floor);

    return 0;
}
```

	Input	Expected	Got	
✔	6 1 2 8 10 12 19 5	2	2	✔
✔	5 10 22 85 108 129 100	85	85	✔
✔	7 3 5 7 9 11 13 15 10	9	9	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Vaibhav Kiran R 251801363-2025-AIDS V2

Started on	Thursday, 27 August 2026, 11:20 AM
State	Finished
Completed on	Thursday, 27 August 2026, 11:22 AM
Time taken	2 mins 11 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int binarySearch(int arr[], int low, int high, int target) {
    if (low > high)
        return -1;

    int mid = low + (high - low) / 2;

    if (arr[mid] == target)
        return mid;

    if (arr[mid] > target)
        return binarySearch(arr, low, mid - 1, target);

    return binarySearch(arr, mid + 1, high, target);
}

int findPair(int arr[], int low, int high, int x, int n) {
    if (low >= n)
        return 0;

    int target = x - arr[low];

    // Search only after low to avoid using the same element twice
    if (binarySearch(arr, low + 1, n - 1, target) != -1) {
        printf("%d\n", arr[low]);
        printf("%d\n", target);
        return 1;
    }

    return findPair(arr, low + 1, high, x, n);
}

int main() {
    int n, x;

    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &x);

    if (!findPair(arr, 0, n - 1, x, n)) {
        printf("No");
    }

```

```
51      return 0;
52  }
```

	Input	Expected	Got	
✔	4 2 4 8 10 14	4 10	4 10	✔
✔	5 2 4 6 8 10 100	No	No	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Vaibhav Kiran R 251801363-2025-AIDS V2

Started on	Thursday, 27 August 2026, 11:23 AM
State	Finished
Completed on	Thursday, 27 August 2026, 11:23 AM
Time taken	32 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Write a Program to Implement the Quick Sort Algorithm

Input Format:
The first line contains the no of elements in the list-n
The next n lines contain the elements.

Output:
Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

Answer:

```c
#include <stdio.h>

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pivot = arr[high];
        int i = low - 1;
        int temp;

        for (int j = low; j < high; j++) {
            if (arr[j] <= pivot) {
                i++;

                temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }

        temp = arr[i + 1];
        arr[i + 1] = arr[high];
        arr[high] = temp;

        int p = i + 1;

        quickSort(arr, low, p - 1);
        quickSort(arr, p + 1, high);
    }
}

int main() {
    int n;

    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    quickSort(arr, 0, n - 1);

    for (int i = 0; i < n; i++) {
        printf("%d", arr[i]);
        if (i < n - 1)
            printf(" ");
```

```
47      }
48
49      return 0;
50  }
```

	Input	Expected	Got	
✔	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✔
✔	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✔
✔	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course